四、Multi-Agent

1.Context Engineering

在 Agentic 系统时代，Context Engineering 已成为决定 LLM 应用成败的关键工程能力。

即使模型能力足够强大，如果 Agent 在执行过程中无法持续获得准确、完整且结构良好的上下文，规划、工具调用和反思环节依然容易出错。Context Engineering 正是为解决这一系统性问题而出现的工程实践。

1.1 概述

1.1.1 为什么需要 Context Engineering

在传统的大模型应用中，模型通常只完成一次性的问答任务：

```
用户问题 -> Prompt -> LLM -> 回答
```

此时开发者主要关注 Prompt 是否写得足够清晰，因此 Prompt Engineering 成为提升效果的核心手段。但在 Agent 系统中，情况发生了根本变化：

```
用户任务 -> 任务规划 -> 工具调用 -> 结果分析 -> 再次规划 -> 再次调用工具 -> ……
```

整个过程中，大模型需要不断读取和利用各种信息，包括：

- 用户原始需求
- 历史对话记录
- Memory 记忆
- RAG 检索结果
- Tool 返回结果
- Agent 中间推理状态
- Workflow 当前执行状态

如果这些信息组织不合理，就会出现以下问题：

- 遗忘关键目标
- 重复调用工具
- 上下文过长导致性能下降
- 错误信息不断累积
- 多 Agent 之间信息不一致

此时问题已经不再是 Prompt 写得好不好，而是：**如何让 Agent 在整个执行过程中持续获得正确、完整且高质量的上下文信息?**

为了解决这一问题，Context Engineering 逐渐成为 Agentic 系统时代的重要工程实践。

1.1.2 什么是 Context Engineering

简单来说：

Context Engineering 就是研究“在正确的时间，把正确的信息，以正确的形式，提供给 Agent”的工程方法。

在 LLM 驱动的系统中，上下文不仅仅是 Prompt 本身，还包括用户输入、系统指令、历史消息、Memory、RAG 检索内容、Tool 返回结果以及 Workflow 状态等。Context Engineering 的核心任务，就是系统性地管理这些信息的获取、组织、传递和更新过程，使模型始终工作在高质量上下文环境下。

因此，它关注的已经不再是单个 Prompt，而是上下文在整个系统中的生命周期管理。

1.1.3 典型案例

假设用户提出一个任务：

> 帮我分析最近三个月英伟达股价上涨的原因，并生成一份投资报告。

Agent 的执行过程可能如下：

- 步骤1：搜索英伟达最近三个月新闻
- 步骤2：获取财报数据
- 步骤3：分析股价走势
- 步骤4：生成报告

在执行过程中，每一步都会产生新的信息：新闻结果、财报数据、分析结论、中间规划、最终报告等。如果 Agent 把所有内容无脑塞进上下文，就会导致：

- 上下文越来越长
- 噪声越来越多
- Token 成本越来越高

最终可能出现遗忘任务目标、重复搜索、分析错误、报告内容混乱等问题。

因此系统需要决定：哪些信息保留、哪些信息压缩、哪些信息丢弃、哪些信息传递给下一步。这正是 Context Engineering 要解决的核心问题。

1.1.4 发展历程

Context Engineering 是 Prompt Engineering 在 Agentic 系统普及后的自然演进，主要经历了以下阶段：

Prompt Engineering 阶段（2023–2024） 关注如何写好单个 Prompt，提升模型在单一任务中的表现。

RAG + Tool Calling 阶段（2024–2025） 上下文开始包含外部检索结果和工具返回信息，重点解决如何给模型提供更多外部知识。

Agentic Workflow 阶段（2025 年至今） 上下文在多步执行和多 Agent 协作中持续流动，核心问题转变为如何管理整个系统中的上下文获取、组织、传递与更新。此时 Context Engineering 正式成为独立且关键的工程领域。

1.1.5

Context Engineering 是指在 LLM 驱动的系统中，系统性地设计、组织、传递、更新和优化上下文信息的工程实践。其核心目标是确保 Agent 或模型在多步执行、动态决策的过程中，能够持续获得高质量、结构清晰、信噪比高的上下文，从而做出更准确的判断和行动。

与传统的 Prompt Engineering 不同，Context Engineering 不再局限于单个提示词的撰写，而是关注上下文在整个系统生命周期中的流动与管理。它强调上下文的**动态性**、**多源性**和**可维护性**，是构建可靠 Agentic 应用的关键支撑能力。

1.1.2 发展历程

Context Engineering 的出现是 Prompt Engineering 在 Agentic 系统普及后的自然演进，其发展可分为三个阶段：

第一阶段：Prompt Engineering 阶段（2023–2024 年） 此阶段以单个静态 Prompt 的优化为核心。开发者通过角色设定、指令设计、Few-shot 示例、Chain-of-Thought 等技术，提升模型在单一任务中的表现。上下文主要由用户输入和系统提示词构成，结构相对简单。

第二阶段：RAG 与早期 Agent 阶段（2024–2025 年） 随着检索增强生成（RAG）和工具调用（Tool Calling）的普及，上下文开始包含外部知识、工具返回结果和中间思考过程。此时上下文的规模和复杂度显著增加，单一 Prompt 已难以有效承载所有必要信息，噪声、冗余和信息缺失等问题逐渐显现。

第三阶段：Agentic Workflow 与 Context Engineering 阶段（2025 年至今） Agentic Workflow 的兴起使得上下文在多步执行、多 Agent 协作中持续流动和迭代。上下文不再是静态输入，而是动态状态的一部分。此时，上下文的质量直接影响 Agent 的规划能力、决策准确性和反思效果。单纯依赖 Prompt 优化已无法满足生产需求，Context Engineering 作为系统性方法应运而生，专注于上下文的全生命周期管理。

这一演进的根本驱动在于：**Agent 的能力上限 increasingly 受限于上下文质量而非模型本身。**当系统从单步生成转向多步动态执行时，上下文的管理方式成为决定系统成败的关键因素。

1. MultiAgent

1.1 背景介绍

1.1.1 单个Agent的边界

（1）**单 Agent 系统在什么情况下会遇到瓶颈?**

我们先通过一个**生活中的类比**来感受单 Agent 的实际局限。

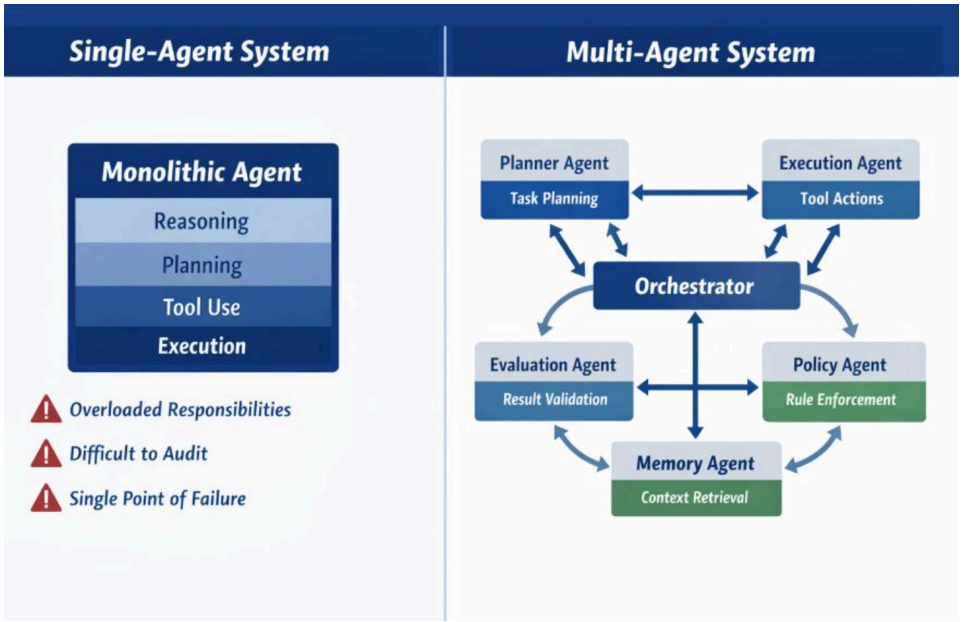
想象一下：让你**一个人独立完成一份《量子计算对金融业影响》的市场调研报告**（要求包含最新技术进展、行业案例、风险分析和投资建议）。即使你能力很强，也会遇到以下现实问题：

- 你必须同时扮演研究员、作家、数据分析师、审稿人多个角色，认知负担极重；
- 当你写到"高盛已大规模部署量子计算机"时，你很难发现自己引用的是两年前的过时信息；
- 你写完初稿后，很难用"旁观者"的视角发现自己逻辑跳跃、结构混乱、某个论点缺乏支撑；
- 一旦发现重大错误，往往只能从头重写，效率极低。

这正是单 Agent 在复杂任务中面临的真实瓶颈：

- **认知负载过重**：同时处理多个角色，容易注意力分散；
- **知识与视角局限**：受单一模型的上下文和知识边界限制，难以覆盖多领域深度；
- **自我纠正能力有限**：虽然 Self-Ask 等机制能让 Agent 进行一定程度的自我反思，但本质上仍是"自己问自己"，难以实现真正客观、跨视角的审核；
- **无法并行与全局优化**：所有步骤只能顺序执行，错误容易累积，无法像团队一样分工协作、互相纠错。

结论： 单个 Agent 适合**简单、边界清晰、实时性要求高**的任务。当任务进入**多领域、多步骤、需要高准确性和可靠性的复杂场景**时，单一 Agent 就会达到能力边界。这时，**Multi-Agent 系统**（多个 Agent 分工协作）就成为必然且高效的解决方案 —— 这就好比从"一个人单干"升级为"组建专业团队协作"。



1.1.2 Multi-Agent 介绍

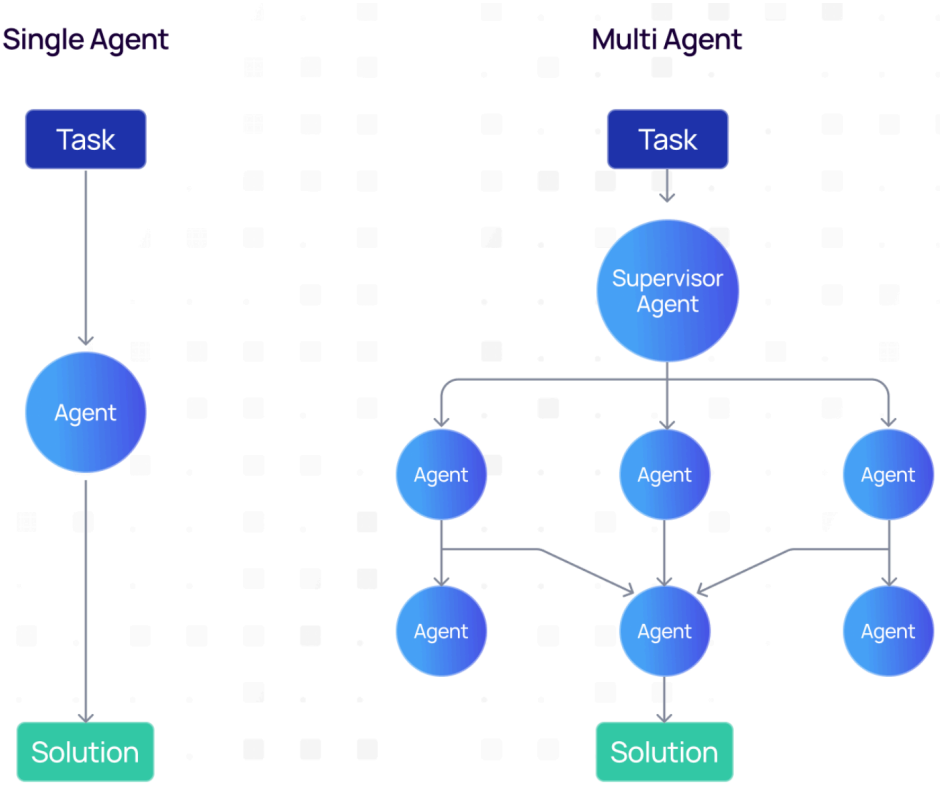
想象一下同样的任务——撰写一份《量子计算对金融业影响》的市场调研报告。

如果你不是一个人单干，而是**组建一个专业团队**：

- **研究员**：只负责查找最新技术进展和真实行业案例
- **分析师**：只负责提炼核心观点和风险评估
- **作家**：只负责把内容写成流畅、专业、可读的报告
- **审核员**：只负责从外部视角检查事实错误、逻辑漏洞和表达问题

这时会发生什么？ 团队成员各司其职、互相配合，最终产出的报告质量远超一个人单干的结果。这就是 **Multi-Agent 系统**的核心思想：

分工 + 协作 + 专业化 → 1+1 > 2



与单 Agent 的本质区别：

- **引入"社会性"**：Agent 之间不再孤立，而是形成了角色、信任、协商等关系
- **产生涌现行为**：多个 Agent 协作后，会出现单个 Agent 无法预测的整体智能（既可能是群体智慧，也可能是协调问题）
- **从"个体智能"转向"群体智能"**：不再依赖单个模型的超强能力，而是依靠团队的协同机制

生活类比： 单 Agent 就像"一个人既当老板又当员工还要自己审稿"，容易疲于奔命； Multi-Agent 就像"组建一个专业团队，每个人专注自己最擅长的部分，最后一起交付高质量成果"。

结论：当任务复杂度超过单个 Agent 的可靠边界时，Multi-Agent 系统就成为提升质量、降低风险、实现规模化智能的最有效途径。

1.1.3 学习目标

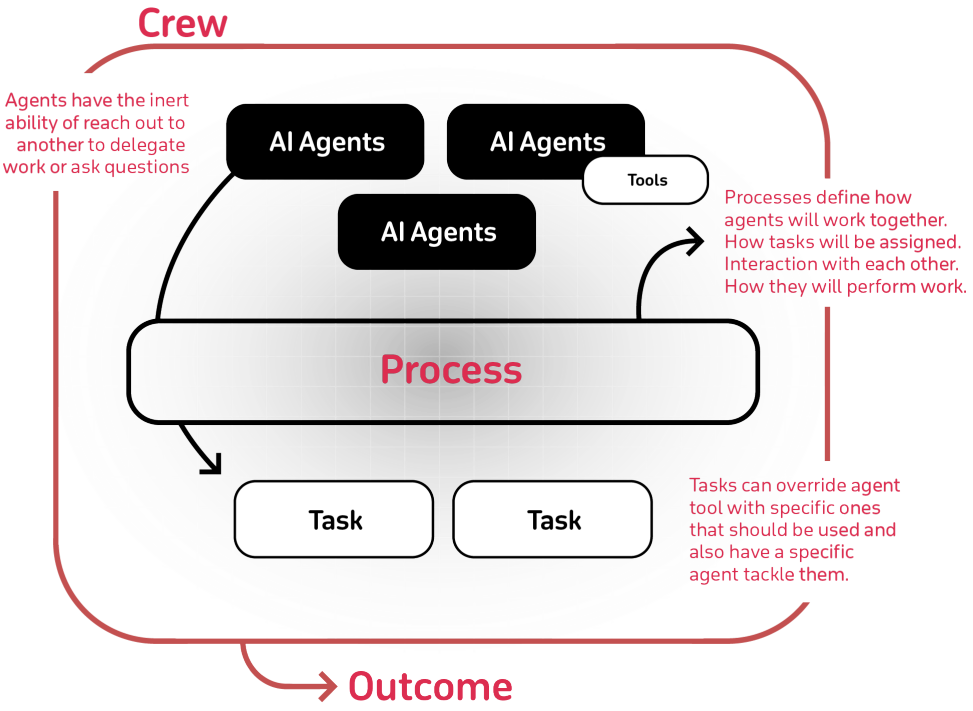
- 理解 Multi-Agent 特有的核心概念（社会性、涌现行为、交互模式、协调机制等）
- 掌握 CrewAI 框架的多 Agent 协作流程（Agent 角色设计、Task 编排、Process 控制）
- 能够识别并解决 Multi-Agent 系统中的常见问题（如角色冲突、信息过载、协调低效）

1.1.4 主流框架

目前业界主流的 Multi-Agent 框架主要有以下三种，各有侧重：

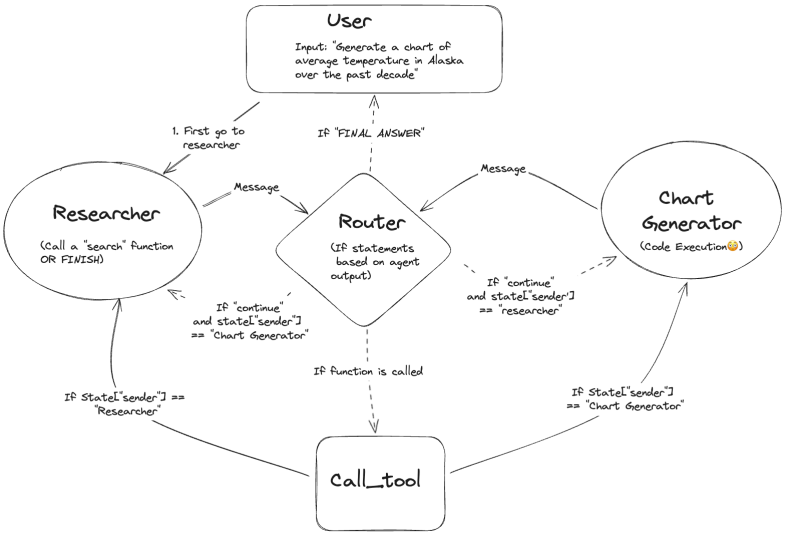
1. CrewAI

- **定位**：团队编排器（Team Orchestrator）
- **特点**：声明式、易上手、适合固定协作流水线
- **优势**：角色定义清晰、Task 依赖简单、适合快速原型开发



2. LangGraph

- **定位**：图式工作流引擎（Graph-based Workflow）
- **特点**：高度灵活、可自定义循环、条件分支、状态持久化
- **优势**：适合复杂动态场景（如需要反思、多轮对话、实时调整的系统）



3. 其他主流框架（简要对比）

目前业界主流的 Multi-Agent 框架主要有以下几种，以下表格从**LangChain 兼容性、学习曲线、上手难度、使用场景**等维度进行了全面对比：

框架名称	LangChain 兼容性	学习曲线	上手难度	使用场景	核心优势
CrewAI	★★★★★（原生完美兼容）	低	★☆☆☆（极易）	固定协作流水线、标准化业务流程（如报告生成、客服、内容审核）	声明式配置、角色清晰、快速上手
LangGraph	★★★★★（LangChain 官方子模块）	中高	★★★☆（中等偏高）	复杂动态对话、反思循环、多轮决策、状态持久化场景	高度灵活、可自定义图、支持条件分支
AutoGen	★★★☆☆（可集成，但需适配）	中	★★☆☆	企业级分布式系统、跨语言协作、多 Agent 大规模交互	事件驱动、支持多语言、分布式能力强
MetaGPT	★★☆☆☆（部分兼容）	中	★★☆☆	完整软件开发流程（产品经理→架构师→程序员）	一站式角色扮演、模拟真实公司分工
OpenDevin	★★★☆☆（可集成）	中高	★★★★	代码调试、软件工程、自主开发任务	沙盒执行环境、IDE集成能力强

- CrewAI 使用的是**声明式**配置（定义 Agent + Task + Process 即可），只需要理解“谁做什么”即可上手。
- LangGraph 则要求理解**图数据结构**（节点、边、状态机、条件边、循环等），这本身就是一个需要一定抽象思维的知识点，对很多学生来说是明显的门槛。
- 虽然 LangChain 官方提供了很多示例，但真正掌握 LangGraph 的状态管理、动态路由和持久化机制，仍

需要一定的实践和调试经验。

框架选择建议：

- **想最快上手、快速看到效果 → 强烈推荐 CrewAI**（上手难度最低，适合初学者）。
- **需要高度灵活的动态流程、复杂决策逻辑** → 选择 **LangGraph**（上手难度更高，但能力上限也更高）。
- **企业级分布式或跨语言场景** → 考虑 **AutoGen**。

教学小结：

CrewAI 像"搭积木"，适合快速搭建团队；**LangGraph** 像"搭电路"，需要理解底层结构，但灵活性更强。大多数实际项目中，开发者会**先用 CrewAI 验证想法，再用 LangGraph 做复杂优化。**

1.2 MultiAgent核心概念

1.2.1 多 Agent 系统的额外维度

与单 Agent 相比，Multi-Agent 系统不再是一个孤立的"个体智能"，而是一个拥有社会性与系统级特性的"群体智能"。这种转变带来了四个重要的额外维度，分别是控制、知识、目标、行为。下面我们逐一展开说明。

（1）控制维度

单 Agent：采用**中心化推理**，所有决策都由同一个模型独立完成。

Multi-Agent：采用**分散或层次化决策。**

- 可以设置一个 **Manager Agent**（经理）负责统筹任务分配、调度和最终决策；
- 也可以让多个 Agent 进行**分布式自治**，通过协商自主决定下一步行动。

实际意义：控制维度的变化让系统更灵活，也更接近真实人类团队的决策方式，但同时需要设计合理的协调机制，否则容易出现决策混乱。

（2）知识维度

单 Agent：只有一个**共享记忆**，所有信息都集中存储。

Multi-Agent：可以同时存在**共享记忆**与**私有记忆**（信息隔离）。

- **共享记忆**：团队共同知识，所有 Agent 都可以读取；
- **私有记忆**：每个 Agent 拥有自己专属的知识库，避免信息污染。

实际意义：知识维度让系统既能实现信息共享，又能保护敏感信息和防止错误信息扩散，是 Multi-Agent 系统实现"专业化"的重要基础。

（3）目标维度

单 Agent：通常只有一个**单一整体目标。**

Multi-Agent：可以同时存在**独立子目标、共同目标，甚至冲突目标。**

- 例如：研究员 Agent 的目标是"找到最准确的信息"，而作家 Agent 的目标是"写出最流畅的文章"，两者需要协调才能达成整体报告目标。

实际意义：目标维度的引入让系统能够处理更复杂的现实任务，但也带来了**目标冲突**的风险，需要设计全局协调机制来平衡局部利益与整体目标。

（4）行为维度

单 Agent：只能**顺序执行**任务。

Multi-Agent：支持**并发执行 + 依赖同步**。

- 多个 Agent 可以同时工作（并行），大大提升效率；
- 但任务之间存在依赖关系时，需要通过同步机制保证正确顺序（如研究员完成后再交给作家）。

实际意义：行为维度是 Multi-Agent 系统效率提升的核心来源，但也对通信和协调能力提出了更高要求。

这四个额外维度（控制、知识、目标、行为）共同构成了 Multi-Agent 系统与单 Agent 的本质区别。理解它们，是我们后续学习社会性、涌现行为、交互模式以及 CrewAI 框架的基础。

维度	单 Agent	多 Agent	说明
控制	中心化推理	分散 / 层次化决策	可采用 Manager 统筹或分布式自治
知识	单一共享记忆	共享记忆 vs 私有记忆（信息隔离）	既可共享知识，也支持隐私保护
目标	单一整体目标	独立目标 / 共同目标 / 冲突目标	可能存在子目标与全局目标的协调
行为	顺序执行	并发执行 + 依赖同步	支持并行工作，大幅提升效率

1.2.2 社会性与涌现行为

在控制、知识、目标、行为四个维度共同作用下，Multi-Agent 系统产生了两个更高层次的核心特性：

（1）**社会性（Sociality）**

社会性是指 **Agent 之间不再是孤立的个体，而是形成了类似人类社会的角色、权力、信任和互动关系。**

具体表现包括：

- **角色分工**：不同 Agent 被赋予明确的身份（如研究员、作家、审核员、经理等）
- **权力关系**：某些 Agent 可能拥有更高的决策权（如 Manager Agent 可以分配任务）
- **信任机制**：Agent 会判断是否相信其他 Agent 提供的信息，而不是盲目接受

生活类比： 单 Agent 就像一个人独自工作；Multi-Agent 就像一个真正的团队——有分工、有层级、有信任，也有互相监督。

（2）**涌现行为（Emergent Behavior）**

涌现行为是指 **多个 Agent 协作后产生的整体系统行为，无法由单个 Agent 单独预测或控制。** 它有两种表现形式：

- **积极的群体智慧（Positive Emergence）**

  多个 Agent 协作后，系统整体表现出了远超单个 Agent 的智能。

  示例：三个 Agent 分别负责研究、写作、校验，最终产出的报告质量、深度和准确性明显高于任何一个 Agent 单独完成的结果。这就是典型的“1+1>2”的群体智慧。

- **消极的任务死锁（Negative Emergence）**

  多个 Agent 协作后，反而出现了单个 Agent 不会出现的问题，导致系统整体性能下降。

示例：两个客服 Agent（A 负责退货，B 负责投诉）在处理同一个客户时，互相推诿责任、重复询问相同问题，最终导致用户体验极差，甚至任务无法完成。这就是典型的“任务死锁”。

社会性让 Multi-Agent 系统具备了“团队属性”，而涌现行为则是这种团队属性带来的结果——它既可能是系统最强大的地方（群体智慧），也可能是最危险的陷阱（任务死锁）。 理解并合理引导涌现行为，是设计高质量 Multi-Agent 系统的核心挑战之一。

1.2.3 Agent 交互模式

Multi-Agent 之间的交互方式远比我们想象的复杂，并非简单的“合作”或“竞争”就能概括。从**博弈论**的宏观视角来看，所有 Agent 之间的交互都可以归为三大类：**协作型、竞争型、混合/博弈型**。

（1）协作型（Cooperative）

Agent 之间共享相同的子目标，互相支持、共享信息，共同朝着同一个最终结果努力。

核心特点：没有根本利益冲突，强调信息共享、优势互补和团队协作。

典型示例： 写作 Agent、校对 Agent、排版 Agent 共同完成一篇市场调研报告。研究员 Agent 把最新数据传递给写作 Agent，写作 Agent 完成后交给校对 Agent 检查事实和逻辑，最终形成高质量报告。

适用场景：内容生成、研究分析、流程自动化等需要多环节配合的任务。

（2）竞争型（Competitive）

Agent 之间存在资源或收益的竞争，每个 Agent 都试图优化自己的指标，甚至以牺牲其他 Agent 为代价。

核心特点：零和博弈（一方获利意味着另一方受损），Agent 之间可能隐藏信息或采取对抗策略。

典型示例： 多个广告出价 Agent 同时竞争同一个广告位，每个 Agent 都根据实时数据调整出价策略，目标是让自己赢得展示机会并最大化 ROI。

适用场景：竞价拍卖、资源分配、策略博弈等场景。

（3）混合/博弈型（Mixed-Motive）

Agent 之间既存在合作需求，又存在利益冲突，是现实世界中最常见、最复杂的交互模式。

核心特点：合作与竞争并存，Agent 需要根据不同阶段灵活切换策略，既要达成共同目标，又要争取自身最大利益。

典型示例： 供应链中的多个 Agent（供应商 Agent、生产 Agent、销售 Agent），它们需要合作保证按时交付，但同时又会在价格、库存分配、利润分成上进行谈判和博弈。

适用场景：商业谈判、供应链管理、游戏 AI、多方协作项目等。

（4）从博弈论视角的三大类划分：

类型	核心特征	常见交互模式	现实示例
完全合作	共享同一目标（A赢则B也赢）	协同、利他	团队写作、救援系统
完全竞争	零和博弈（A赢则B必输）	对抗、自私	竞价广告、棋类AI
混合动机	合作与竞争并存	协商、策略切换	商业谈判、供应链管理

教学提示： 现实中的 Multi-Agent 系统绝大多数属于**混合动机**模式。这也是设计协调机制时最需要考虑的复杂性来源——既要设计合作规则，又要防范利益冲突带来的风险。

1.2.4 通信与协调机制

Multi-Agent 系统要让多个 Agent 高效协作，必须解决两个核心问题：**Agent 之间如何传递信息（通信）** 和 **Agent 之间如何组织工作（协调）**。这两个机制直接决定了系统的运行效率、稳定性和可扩展性。

1.通信方式

Multi-Agent 系统常用的通信方式主要有三种，每种方式适用于不同的协作需求：

（1）**直接消息（Point-to-Point）**

一个 Agent 直接向另一个或多个指定 Agent 发送消息。这种方式最灵活，Agent 可以点对点精准传递信息，实时性强，内容也可根据对方需求定制。

适用场景：Manager Agent 向某个专家 Agent 分配具体任务，或 Agent 之间需要快速反馈的情况。

（2）**黑板系统（Blackboard / Shared Memory）**

所有 Agent 都可以向一个公共的“黑板”写入信息，也可以随时读取黑板上的内容。信息是公开共享的，所有 Agent 都能看到最新状态。

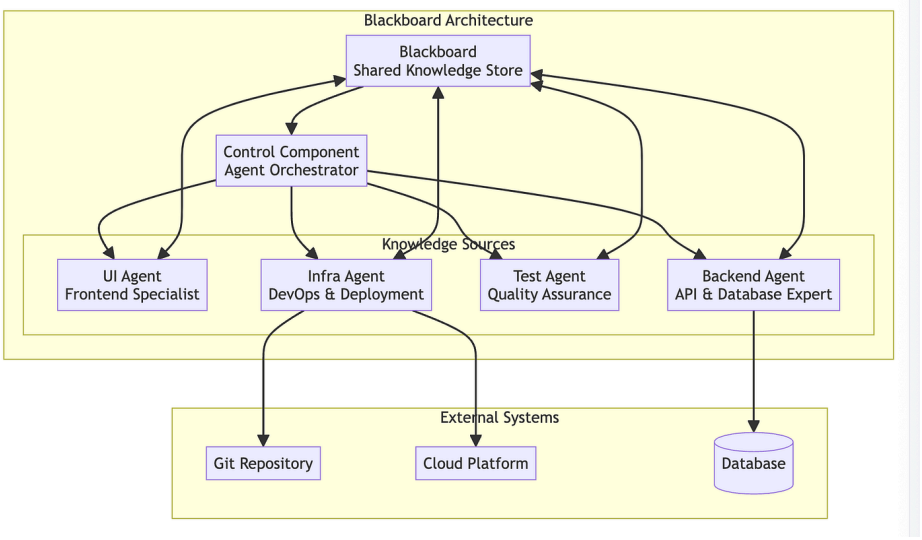
适用场景：需要全体 Agent 共同知晓的项目进度、共享知识库或中间结果。

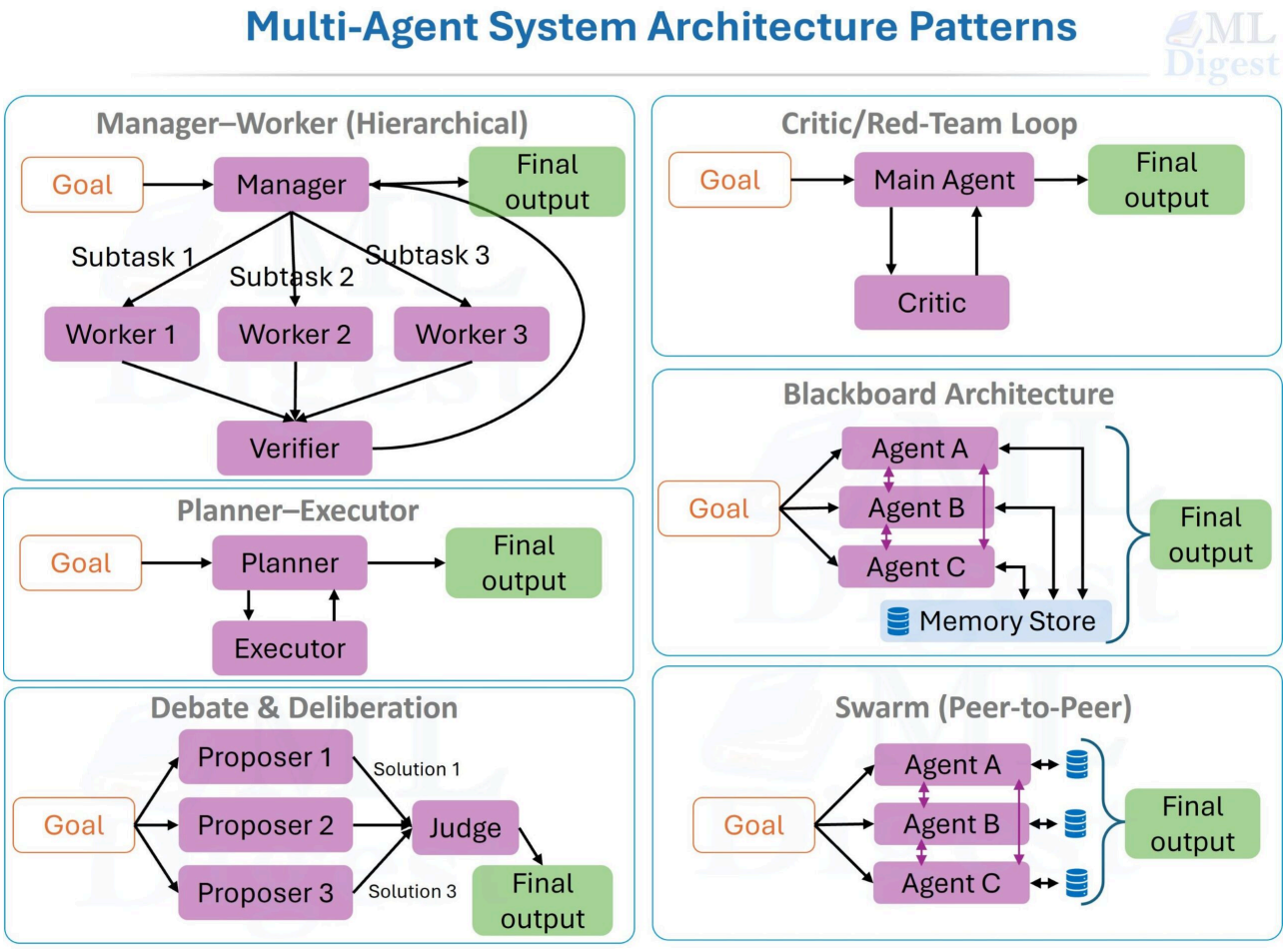
（3）**事件驱动（Publish-Subscribe）**

Agent 发布某种“事件”，其他感兴趣的 Agent 订阅该事件并接收通知。发布者和订阅者之间无需直接知道对方，实现了较好的解耦。

适用场景：动态场景中，当某个 Agent 完成任务后需要自动通知后续 Agent 时。

以下是常见通信方式的示意图：





2.协调架构

通信解决了"信息如何流动"的问题，而协调架构则决定了"谁来指挥整体工作"。主要有两种典型架构：

（1）**集中式协调（Centralized - Manager）**

系统设置一个专门的 **Manager Agent**（经理 Agent），负责全局统筹：分配任务、监控进度、解决冲突、下发指令。其他 Agent 主要负责执行。

优点：管理简单、流程清晰、容易实现。

缺点：Manager 容易成为性能瓶颈和单点故障。

典型框架：CrewAI 默认采用此模式。

（2）**分布式协调（Decentralized）**

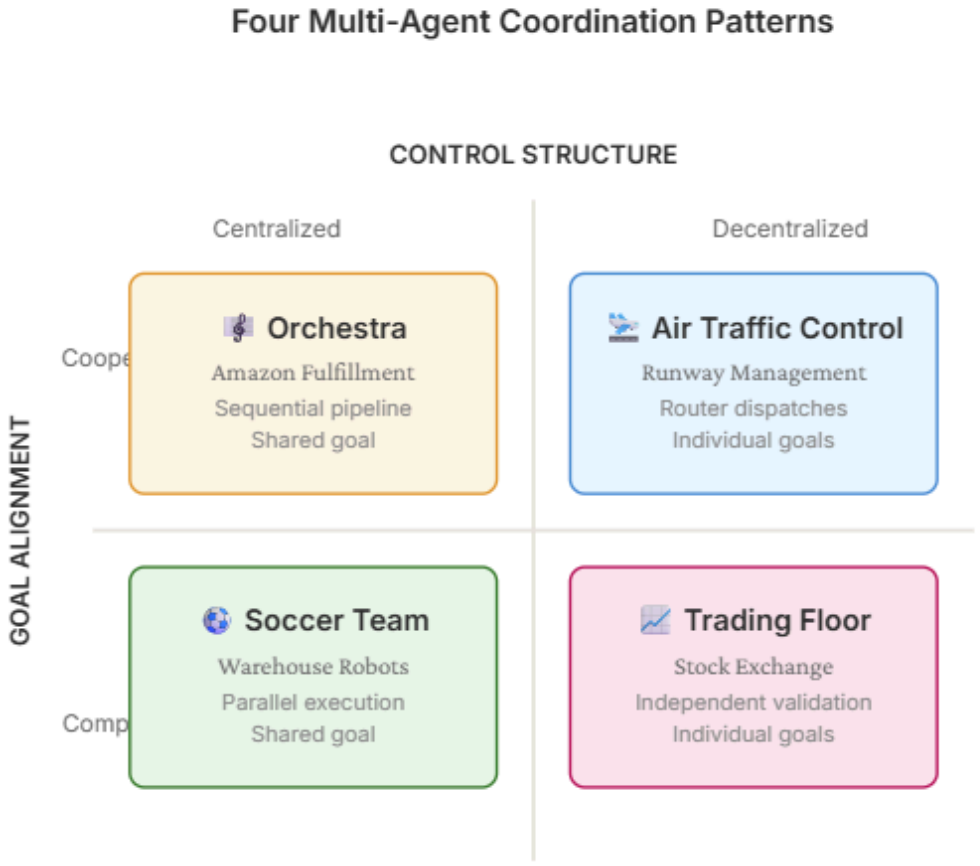
没有中心化的 Manager Agent，各个 Agent 通过协商协议自主沟通、互相协调、共同决策。

优点：扩展性强、无单点故障、容错性高。

缺点：实现复杂、协调开销较大，容易出现死锁或不一致情况。

典型框架：LangGraph 和 AutoGen 更倾向支持此类模式。

3.模式对比



这张图（Four Multi-Agent Coordination Patterns）用**两个维度**来分类 Multi-Agent 系统的协调模式：

（1） Control Structure（控制结构）

系统由谁来主导决策和协调工作。

- **Centralized（集中式）**：存在一个中心化的 **Manager / Orchestrator**（经理/协调者），负责分配任务、监控进度、解决冲突。其他 Agent 主要负责执行。
- **Decentralized（分布式）**：没有中心 Manager，各个 Agent 通过协议自主协商、互相协调、共同决策。

（2）Goal Alignment（目标对齐 / 目标一致性）

Agent 之间的目标是共享的还是各自独立的，甚至存在竞争。

- **Cooperative（合作型）**：所有 Agent 共享同一个整体目标（A 成功则 B 也成功），强调团队协作。
- **Competitive（竞争型）**：Agent 各自有独立目标，甚至存在利益冲突（A 成功可能意味着 B 失败），属于零和或混合博弈。

（3）这两个维度为什么重要?

- **Control Structure（控制结构）** 决定了系统的**管理方式**（集中还是分散），影响系统的**扩展性、鲁棒性**

和实现难度。

- **Goal Alignment（目标对齐）** 决定了 Agent 之间的**利益关系**（合作还是竞争），影响系统的**协调难度和涌现行为。**

（4）把这两个维度组合起来，就形成了四种典型的 Multi-Agent 协调模式（图中四个方格）：

- Centralized + Cooperative → 最常见、最容易实现的模式（CrewAI 默认属于这一类）
- Decentralized + Competitive → 最复杂、最具挑战性的模式

教学小结： 通信方式决定了信息如何在 Agent 之间流动，协调架构决定了谁来指挥整体工作。CrewAI 采用的是**集中式协调 + 任务队列通信**的设计，这种方式结构清晰、实现简单，非常适合教学和快速原型开发；而 LangGraph 等框架则更倾向于灵活的分布式协调，适合需要动态决策的复杂场景。

1.2.5 任务依赖图

在 Multi-Agent 系统中，**通信**解决了"信息如何传递"，**协调**解决了"谁来指挥"，而**任务流转**则解决了"不同任务之间应该如何衔接、顺序和分支"这个核心问题。

任务依赖图（Task Dependency Graph） 就是用来描述多个任务之间执行关系的图形化工具。它明确规定了任务的先后顺序、是否可以并行、以及在什么条件下切换到下一个任务，是 Multi-Agent 系统"工作流程"的蓝图。

简单来说，任务依赖图回答的是：**"谁先做？谁后做？能不能同时做？什么时候切换？"**

Multi-Agent 系统中最常见的三种任务流转模式如下：

1. 串行流转（Sequential）

任务必须**一个接一个**严格按顺序执行，前一个任务完全完成后才能启动后一个任务。

特点：逻辑清晰、依赖关系明确、容易调试和追踪。

示例：研究员 Agent 完成资料收集 → 作家 Agent 才开始撰写初稿 → 校验 Agent 最后进行事实和格式检查。

适用场景：任务之间存在强依赖关系，必须前一步完成才能进行下一步的流程。

2. 并行流转（Parallel）

多个任务可以**同时进行**，不需要互相等待。

特点：显著提升系统整体效率，充分利用计算资源。

示例：研究员 Agent 查找技术资料的同时，市场分析师 Agent 可以同步收集行业案例，最后再将两部分结果合并。

适用场景：任务之间相对独立，可以并行加速完成的场景。

3. 条件分支流转（Conditional）

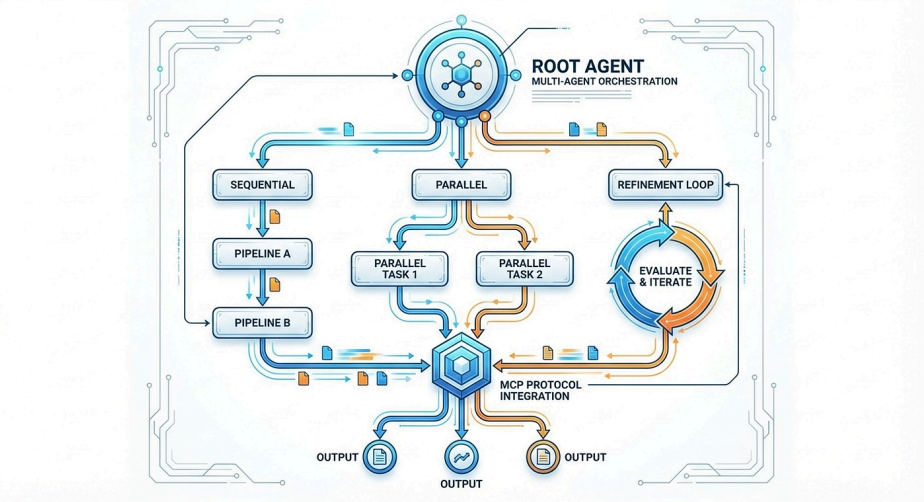
根据当前任务的结果**动态决定**下一步执行哪个任务。

特点：系统具有判断和智能决策能力，更加灵活。

示例：校验 Agent 检查完文章后，如果发现事实错误，就返回给作家 Agent 修改；如果全部通过，则直接输出最终报告。

适用场景：需要根据中间结果进行判断和调整的复杂流程。

以下是典型的任务依赖图示意图，帮助你直观理解三种流转方式：



教学小结：

任务依赖图是 Multi-Agent 系统设计中非常重要的一环。它决定了整个系统的执行效率和灵活性。CrewAI 默认采用较为简单的**串行 + 集中式**依赖图，适合初学者快速上手；而 LangGraph 则可以非常灵活地实现串行、并行、条件分支等复杂依赖关系，适合需要动态决策的高阶场景。

1.2.6 多 Agent 系统特有的挑战与应对

挑战	典型表现	应对策略
协调开销	通信频繁导致决策延迟	减少串行步骤，采用异步执行
角色冲突	两个 Agent 执行相同动作	明确角色提示词 + 任务唯一指派
信息过载	共享记忆污染	设置私有字段，只共享必要信息
目标不一致	Agent 追求局部最优，损害全局	全局 reward 设计（多 Agent 强化学习）

本阶段小结：

Multi-Agent 不是简单“多个 LLM 跑任务”，而是一个**系统设计问题。**

它的强大来自于“分工协作”和“涌现智能”，但也带来了协调、通信和冲突等全新挑战。

理解这些核心概念，是后续熟练使用 CrewAI 框架、设计高效 Multi-Agent 系统的关键基础。

2.CrewAI

CrewAI 是一个开源的多 Agent 协作框架，由 CrewAI 团队开发，专注于帮助开发者**快速构建由多个 Agent 组成的智能团队。**它与 LangChain 生态高度兼容，是目前最受欢迎、最易上手的 Multi-Agent 框架之一。

CrewAI 的主要优势：

- **声明式编程**：不需要编写复杂的控制代码，只需“声明” Agent 做什么、Task 是什么、流程如何，就能快速搭建团队。

- **上手难度低**：即使没有深厚的多 Agent 经验，也能在几分钟内搭建一个可运行的协作系统。
- **角色驱动**：每个 Agent 都有清晰的角色（Role）、目标（Goal）和背景故事（Backstory），让 LLM 更容易扮演好自己的“专家”身份。
- **与 LangChain 完美集成**：可以直接使用 LangChain 的工具、LLM 和内存模块。
- **适合教学和快速原型**：结构清晰、调试友好，非常适合课堂演示和项目初期验证。

本节我们将从以下几个方面系统讲解 CrewAI：

- CrewAI 的四个核心抽象（Agent、Task、Crew、Process）
- 实战案例（如何用代码快速搭建多 Agent 团队）
- 两种主要流程模式（Sequential 和 Hierarchical）
- 常见问题与优化技巧

通过本节学习，你将能够独立使用 CrewAI 构建一个完整的多 Agent 协作系统。

2.1 核心抽象

CrewAI 提供了一套**声明式（Declarative）的抽象层**，让开发者可以用最少的代码快速组建多 Agent 团队。它的设计哲学是“一切皆可声明”——你只需定义好 Agent、Task、Crew 和 Process，就能搭建一个完整、可运行的多 Agent 系统。

下面我们逐一介绍 CrewAI 的四个核心抽象，并配上实际代码示例。

2.1.1 Agent（智能体）

（1）概念

Agent 是 CrewAI 中最基本的执行单元，它封装了一个完整的工作角色。

一个 Agent 主要包含三个关键部分：

- 角色提示词（Role & Backstory）：定义身份和行为风格
- 工具集（Tools）：该 Agent 专属的可调用工具
- 记忆（Memory）：支持短期和长期记忆

（2）**代码示例**：

```
from crewai import Agent
from langchain_core.tools import tool

# 定义搜索工具
@tool
def wikipedia_search(query: str) -> str:
    """使用维基百科搜索信息"""
    # ... (工具实现代码)
    pass

researcher = Agent(
    role="量子计算金融研究员",
    goal="从可靠来源搜索并提炼量子计算对金融业影响的3个核心观点",
    backstory="你是一位严谨的金融科技研究员，擅长从学术和行业资料中提取关键洞见。",
    tools=[wikipedia_search],
```

```
    verbose=True
)
```

（3）对应参数介绍

- role：Agent 的角色名称（决定身份）
- goal：该 Agent 的具体目标（指导行为）
- backstory：背景故事（帮助 LLM 更好地扮演角色）
- tools：该 Agent 可使用的工具列表
- verbose：是否打印详细思考过程（调试用）

2.1.2 Task（任务）

（1）概念介绍

Task 是 CrewAI 中具体的工作单元，它描述了一个明确、可执行的任务。

一个 Task 主要包含：

- 任务描述（Description）
- 预期输出格式（Expected Output）
- 依赖关系（Dependencies）：可指定前置 Task

（2）**代码示例**：

```
from crewai import Task

task1 = Task(
    description="搜索量子计算对金融业的影响，提炼出3个最核心、最具代表性的观点，并列出简要理由。",
    expected_output="3个核心观点，每个观点包含标题和1-2句解释",
    agent=researcher
)
```

（3）对应参数介绍

- description：任务的具体描述
- expected_output：期望的输出格式（强烈建议明确写出）
- agent：指定执行该任务的 Agent
- context：（可选）前序任务的输出会自动传入，作为上下文

2.1.3 Crew（团队）

（1）概念介绍

Crew 是整个多 Agent 系统的“容器”，它负责把多个 Agent 和多个 Task 组织在一起运行。

Crew 的主要职责是：

- 统一管理所有 Agent

- 编排所有 Task 的执行流程
- 支持串行、并行、层次化等多种执行模式

（2）**代码示例**：

```
from crewai import Crew, Process

crew = Crew(
    agents=[researcher, writer, validator],
    tasks=[task1, task2, task3],
    process=Process.sequential,    # 顺序执行
    verbose=2                      # 显示详细执行日志
)
```

（3）对应参数介绍

- agents：所有 Agent 的列表
- tasks：所有 Task 的列表（按执行顺序排列）
- process：协作流程（Sequential 或 Hierarchical）
- verbose：全局日志级别（2 表示显示详细执行过程）

2.1.4 Process（流程）

（1）概念介绍

Process 定义了团队的协作方式，即任务如何被执行。

CrewAI 目前主要支持两种流程：

- **Sequential（顺序流程）**：任务按顺序依次执行，最简单、最可靠。
- **Hierarchical（分级流程）**：引入 Manager Agent 统筹分配任务，下级 Agent 执行，适合更复杂的场景。

（2）**代码示例**：

```
# 顺序流程（最常用）
crew = Crew(
    agents=[researcher, writer, validator],
    tasks=[task1, task2, task3],
    process=Process.sequential
)

# 分级流程（更灵活）
crew = Crew(
    agents=[manager, researcher, writer, validator],
    tasks=[task1, task2, task3],
    process=Process.hierarchical,
    manager_agent=manager    # 指定经理 Agent
)
```

(3) 对应参数介绍

- process=Process.sequential：任务按顺序依次执行（简单可靠）
- process=Process.hierarchical：引入 Manager Agent 统筹分配任务，下级 Agent 执行（更灵活，适合复杂场景）

教学小结：

CrewAI 的核心设计哲学是**"一切皆可声明"**。开发者只需要定义好 Agent（角色）、Task（工作）、Crew（团队）和 Process（流程），就能快速搭建一个完整、可运行的多 Agent 系统。这种声明式的方式极大降低了学习和开发门槛。

2.2 CrewAI 的通信与记忆模型

2.2.1 通信方式

(1) 概念介绍

CrewAI 的通信方式以**简单且显式**为核心原则，主要通过 **Task 的 `context` 字段** 实现任务间的数据传递。前一个 Task 的输出结果会自动作为后一个 Task 的上下文（Context）传入，从而实现清晰、可控的信息流动。这种设计避免了隐式依赖导致的混乱，非常适合教学和工程实践。

(2) 示例代码

```
task1 = Task(
    description="搜索量子计算对金融业的影响，提炼3个核心观点",
    expected_output="3个核心观点，每个观点包含标题和解释",
    agent=researcher
)

task2 = Task(
    description="基于研究员提供的核心观点，撰写一篇约500字的短文",
    expected_output="一篇完整的500字短文",
    agent=writer,
    context=[task1]         # 关键：自动接收 task1 的输出作为上下文
)

task3 = Task(
    description="检查文章的事实、逻辑和可读性",
    expected_output="最终优化后的文章 + 修改说明",
    agent=validator,
    context=[task2]         # 接收 task2 的输出
)
```

(3) 特点与说明

- 通信是**显式**的：通过 `context` 参数明确指定依赖关系，数据流向一目了然。
- 自动传递：前序 Task 的输出会自动注入后序 Task 的上下文，开发者无需手动管理。
- 优点：简单、直观、易调试，极大降低了 Multi-Agent 协作的复杂度。

2.2.2 记忆模型

（1）概念介绍

CrewAI 提供了丰富且灵活的记忆机制，让每个 Agent 能够“记住”历史信息，从而实现更智能的协作。记忆模型是 Multi-Agent 系统实现连续性和上下文感知的关键。

（2）示例代码（记忆相关配置）

```python
from crewai import Agent
from crewai.memory import ShortTermMemory, LongTermMemory

researcher = Agent(
    role="量子计算金融研究员",
    goal="...",

    # 短期记忆（默认开启）
    memory=True,                    # 启用记忆功能

    # 长期记忆（向量数据库）
    long_term_memory=LongTermMemory(
        storage="chroma",           # 可选 chroma / faiss 等
        collection_name="research_memory"
    ),

    # 短期记忆配置
    short_term_memory=ShortTermMemory(
        max_recent_interactions=10  # 保留最近10次交互记录
    ),

    verbose=True
)
```

（3）详细介绍

CrewAI 的记忆模型主要分为以下三种类型：

- **短期记忆（Short-term Memory）**

  保存当前对话历史、最近几次交互记录以及任务过程中的中间结果。

  类似于人类的“工作记忆”，用于处理当前正在进行的任务。

  特点：实时性强，容量较小，通常只保留最近几次交互。

- **长期记忆（Long-term Memory）**

  基于向量数据库实现，可持久化存储重要知识、历史任务结果和领域知识。

  即使 Agent 重启或任务结束，长期记忆依然存在。

  特点：容量大、可跨任务复用，适合需要长期积累知识的场景。

- **共享记忆 vs 私有记忆**

  共享记忆：默认情况下，所有 Agent 都可以读取团队的公共记忆（例如项目整体进度、共同知识库）。

  私有记忆：可以通过配置让某个 Agent 拥有自己专属的记忆，其他 Agent 无法访问（防止信息污染）。

注意事项与风险：

- 共享记忆虽然方便，但容易导致**信息过载**和**信息污染**（一个 Agent 的错误信息被其他 Agent 误用）。
- 实际开发中建议：**只共享必要信息**，对敏感或易出错的内容使用私有记忆。
- 合理控制记忆容量，避免上下文过长导致 LLM 成本上升或幻觉增加。

教学小结：

CrewAI 的通信方式以显式 `context` 为主，简单可靠；记忆模型则提供了短期、长期、共享与私有的灵活组合，让 Agent 既能"记住过去"，又能有效控制信息流动。这些机制共同构成了 CrewAI 高效协作的基础。

2.3 Crew AI 的实现

2.3.1 项目背景介绍

（1）量子计算介绍

量子计算是当前最受关注的前沿科技之一。它利用量子比特（qubit）的叠加和纠缠特性，能够在某些特定问题上实现指数级计算速度的提升，被誉为"下一代计算革命"。

金融行业是量子计算最有可能率先落地的领域之一。因为金融业务中存在大量高度复杂的计算难题，例如：

- 期权定价
- 投资组合优化
- 风险模拟
- 欺诈检测
- 高频交易策略

传统计算机需要花费数十年才能完成的计算，量子计算机理论上可能在几分钟甚至几秒内完成。这不仅会深刻改变金融机构的决策方式，还可能重塑整个行业的竞争格局。

（2）**项目任务**： 请 Multi-Agent 团队协作撰写一篇**约 500 字的短文**，主题为《量子计算对金融行业的影响》。文章需要包含：

- 量子计算的核心优势
- 对金融行业的实际影响
- 潜在风险与挑战
- 未来发展趋势

（3）**为什么用这个项目来学习 CrewAI?**

- 它是一个**典型的多领域复杂任务**：需要同时涉及科技知识、金融知识、逻辑分析和文字表达；
- 单 Agent 很难同时做好"研究准确性"和"表达专业性"，容易出现事实错误或结构混乱；
- 而 Multi-Agent 系统可以通过分工协作（研究员专注查证、作家专注表达、校验 Agent 专注把关），显著提升输出质量。

这个项目既能帮助我们理解量子计算这一前沿技术，又能直观体会 **Multi-Agent 在处理复杂任务时的优势** ——这正是我们学习 CrewAI 的核心目的。

使用 **Multi-Agent 系统**，我们可以组建一个专业团队：

- **研究员 Agent** 专注查找最新可靠信息

- **作家 Agent** 专注将观点转化为流畅文章
- **校验 Agent** 专注事实审核和最终优化

通过分工协作，系统能够显著提升文章的质量和可靠性。这也是我们本章要重点实践的项目。

项目目标： 通过 CrewAI 框架，让三个 Agent 协同完成这篇短文，体会 Multi-Agent 在真实任务中的协作流程、通信机制和优势。

（4）安装依赖

在开始使用 CrewAI 之前，需要安装以下核心依赖：

```
pip install crewai
pip install langchain-core langchain-community wikipedia
```

- `crewai`：核心框架
- `wikipedia`：用于实现搜索工具（无需 API Key）

2.3.2 项目思路

（1）**项目核心**

本项目的核心目标是：**通过 CrewAI 框架，让多个 Agent 组成一个专业团队，协作完成一篇高质量的《量子计算对金融行业的影响》500 字短文。**

量子计算是当前最具革命性的前沿技术之一，它对金融行业的期权定价、风险管理、投资组合优化等复杂计算问题具有巨大潜力。但要写好这样一篇文章，需要同时具备**科技深度、金融视野、逻辑分析和专业表达**等多方面能力。单靠一个 Agent 很难同时做好，而 Multi-Agent 系统可以通过明确分工、协作和审核，大幅提升输出质量。

（2）**团队分工规划**

我们将组建一个**三 Agent 协作团队**，每个 Agent 专注自己最擅长的环节：

- **研究员 Agent**：负责收集最新可靠信息，提炼 3 个核心观点（科技原理 + 金融应用 + 风险挑战）
- **作家 Agent**：基于研究员提供的观点，将内容组织成结构清晰、语言流畅的 500 字短文
- **校验 Agent**：对最终文章进行事实核查、逻辑审查和表达优化，确保专业性和可读性

整体协作流程：研究员 → 作家 → 校验（顺序执行），通过 Task 的 `context` 实现信息自动传递。

（3）**在开始 Multi-Agent 项目之前，同学们需要做的事情**：

- **明确项目目标**

  清晰定义最终交付物（字数、内容要求、质量标准），避免后期反复修改。

- **进行任务拆分**

  将复杂任务分解成多个独立但相互关联的子任务，确定每个子任务由哪个角色负责。

- **设计 Agent 角色**

  为每个 Agent 定义清晰的 `role`、`goal` 和 `backstory`，让 LLM 能够准确扮演对应专家。

- **规划任务依赖关系**

  决定是采用顺序执行、并行执行还是条件分支，提前设计好 Task 之间的 `context` 传递。

- **准备必要工具**

根据任务需求提前封装或引入所需工具（如搜索工具、计算工具等）。

- **定义输出标准**

  为每个 Task 明确 `expected_output`，便于后续校验和评估。

教学小结：

一个好的 Multi-Agent 项目，**70% 的工作量在于前期规划。**清晰的项目思路、合理的团队分工和严谨的任务拆分，是项目成功的关键。这也是我们从“会用 CrewAI”走向“会设计 Multi-Agent 系统”的重要一步。

2.3.3 完整代码实现

```python
from crewai import Agent, Task, Crew, Process
from langchain_core.tools import tool
import wikipedia

# ==================== 项目说明 ====================
# 项目：撰写一篇《量子计算对金融行业的影响》500字短文
# 团队分工：研究员 → 作家 → 校验（顺序执行）

# ==================== 1. 定义工具 ====================
@tool
def wikipedia_search(query: str) -> str:
    """使用维基百科搜索信息（支持中文）"""
    try:
        wikipedia.set_lang("zh")
        if "维基百科" in query:
            query = query.replace("维基百科", "").strip()
        page = wikipedia.page(query, auto_suggest=False)
        return f"标题: {page.title}\n摘要: {page.summary[:800]}..."
    except Exception as e:
        return f"搜索错误: {str(e)}"

# ==================== 2. 定义三个 Agent ====================
researcher = Agent(
    role="量子计算金融研究员",
    goal="从可靠来源搜索并提炼量子计算对金融业影响的3个核心观点",
    backstory="你是一位严谨的金融科技研究员，擅长从学术和行业资料中提取关键洞见。",
    tools=[wikipedia_search],
    verbose=True
)

writer = Agent(
    role="专业金融科技作家",
    goal="基于研究员提供的核心观点，撰写一篇流畅、结构清晰的500字短文",
    backstory="你是一位经验丰富的金融科技作者，文笔简洁专业，擅长将技术概念转化为通俗易懂的文字。",
    verbose=True
)

validator = Agent(
    role="内容校验专家",
    goal="检查文章的事实准确性、逻辑连贯性和可读性，并给出最终优化版本",
```

```
    backstory="你是一位严苛的内容审核专家，对事实错误和表述问题零容忍。",
    verbose=True
)

# ==================== 3. 定义三个 Task（带依赖关系） ====================
task1 = Task(
    description="搜索量子计算对金融业的影响，提炼出3个最核心、最具代表性的观点，并列出简要理由。",
    expected_output="3个核心观点，每个观点包含标题和1-2句解释",
    agent=researcher
)

task2 = Task(
    description="基于研究员提供的3个核心观点，撰写一篇约500字的短文，要求结构清晰、语言专业且通俗易懂。",
    expected_output="一篇完整的500字左右的短文（包含标题）",
    agent=writer,
    context=[task1]           # 自动接收 task1 的输出作为上下文
)

task3 = Task(
    description="检查文章中的事实错误、逻辑问题、格式和可读性，给出最终优化后的版本。",
    expected_output="最终优化后的完整短文 + 简要修改说明",
    agent=validator,
    context=[task2]           # 自动接收 task2 的输出
)

# ==================== 4. 组建 Crew 并运行 ====================
crew = Crew(
    agents=[researcher, writer, validator],
    tasks=[task1, task2, task3],
    process=Process.sequential,   # 顺序执行：研究员 → 作家 → 校验
    verbose=2                     # 2 表示显示详细执行日志
)

# 启动执行
print("🚀 开始执行 CrewAI 多 Agent 协作任务...\n")
result = crew.kickoff()

print("\n" + "="*60)
print("最终输出结果")
print("="*60)
print(result)
```

2.3.4 运行说明与输出分析

运行上述代码后，CrewAI 会输出详细的执行日志。你会看到：

- **Task 1**：研究员 Agent 调用 `wikipedia_search` 工具，输出 3 个核心观点
- **Task 2**：作家 Agent 自动接收 Task 1 的输出作为上下文，撰写完整文章
- **Task 3**：校验 Agent 接收 Task 2 的文章，进行最终检查和优化

关键观察点：

- 通过 `context=[taskX]` 实现**显式数据传递**，任务间信息流动清晰可见
- `verbose=2` 可以看到每个 Agent 的思考过程、工具调用和输出
- 整个流程是**顺序执行**的，符合人类团队的协作逻辑

教学小结：

通过这个完整案例，你可以看到 CrewAI 的核心抽象（Agent、Task、Crew、Process）如何组合成一个可运行的多 Agent 系统。下一节我们将学习 Hierarchical（分级）流程，并对比 Sequential 模式的差异。

2.4 对比实验

2.4.1 实验目的介绍

为了直观感受 **单 Agent** 与 **Multi-Agent（CrewAI）** 在实际任务中的差异，我们设计了一个对比实验。

实验任务：

使用完全相同的主题——**《量子计算对金融行业的影响》**，撰写一篇约 500 字的短文。

实验目的：

- 对比单 Agent（LangChain `create_agent`）和多 Agent（CrewAI）在**内容准确性、结构清晰度、专业性、可读性**等方面的表现差异
- 通过客观的评分机制，量化两种方式的优劣
- 帮助同学们理解：**什么时候适合使用单 Agent，什么时候需要切换到 Multi-Agent**

2.4.2 实验设置

方案 A：单 Agent（使用 `create_agent` 模式）

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
import wikipedia

@tool
def wikipedia_search(query: str) -> str:
    """使用维基百科搜索信息"""
    # ...（与之前相同的工具实现）
    pass

llm = ChatOpenAI(temperature=0.7, model="deepseek-chat")

single_agent = create_agent(
    model=llm,
    tools=[wikipedia_search],
    system_prompt="""你是一位专业的金融科技作家。
请独立完成以下任务：撰写一篇约500字的短文，主题为《量子计算对金融行业的影响》。
要求：
- 包含至少3个核心观点
```

```
- 结构清晰（引言、主体、结论）
- 语言专业且通俗易懂
- 直接输出完整文章""",
    debug=False
)

# 执行
result_single = single_agent.invoke({"input": "请撰写一篇量子计算对金融行业影响的500字短文"})
print(result_single["output"])
```

方案 B：多 Agent（CrewAI）

使用我们在 2.3.3 中已经搭建好的 **研究员** → **作家** → **校验** 三 Agent 团队（顺序执行），直接运行 `crew.kickoff()`。

2.4.3 常见评分指标

为了让单 Agent 与 Multi-Agent 的对比更加客观、科学，我们采用一套**生成式 AI 常用评估指标**。这些指标参考了 RAG（Retrieval-Augmented Generation）等领域的标准评估体系，同时结合短文写作的特点，设计了以下5个核心维度（每个维度满分10分，总分50分）。

1. 忠实度（Faithfulness）

- **定义**：文章内容是否忠实于可靠的事实和来源，没有幻觉（Hallucination）或编造信息。
- **重点考察**：引用的技术事实、数据、案例是否准确，是否出现过时信息或虚假结论。
- **评分要点**：高分要求所有观点都有可验证的依据；低分表现为明显的事实错误或无来源推测。
- **为什么重要**：生成式模型最容易出现“编造事实”的问题，忠实度是评估可靠性的核心指标。

2. 相关性（Relevancy）

- **定义**：文章内容是否紧密围绕主题“量子计算对金融行业的影响”，没有偏题或无关内容。
- **重点考察**：是否真正回答了“量子计算如何影响金融行业”，而不是泛泛谈论量子计算或金融。
- **评分要点**：高分要求所有段落都服务于主题；低分表现为大量无关背景介绍或跑题内容。

3. 结构清晰度（Coherence & Structure）

- **定义**：文章逻辑是否连贯、层次是否分明、结构是否完整。
- **重点考察**：是否有清晰的引言、主体分层（技术原理、行业影响、风险、建议）和结论，段落之间过渡是否自然。
- **评分要点**：高分要求逻辑流畅、层次清晰；低分表现为跳跃、重复或结构混乱。

4. 专业性（Professionalism & Depth）

- **定义**：文章的专业深度、术语准确性和分析水平。
- **重点考察**：是否准确使用量子计算和金融领域的专业术语，是否提供了有深度的洞见，而非表面描述。
- **评分要点**：高分要求既有技术解释，又有金融影响的分析；低分表现为概念模糊或停留在表面。

5. 可读性（Readability）

- **定义**：文章语言是否流畅、通俗易懂、适合目标读者阅读。
- **重点考察**：句子是否简洁、表述是否自然、是否避免过于生涩的专业术语堆砌。
- **评分要点**：高分要求非专业读者也能轻松理解；低分表现为生硬、晦涩或过于口语化。

评分方式： 由一个独立的**评分大模型（Judge LLM）** 同时阅读两篇文章，严格按照以上5个维度打分，并输出 JSON 格式的结果，便于后续统计和对比。

教学小结： 这些指标（忠实度、相关性、结构清晰度、专业性、可读性）是目前生成式 AI 内容评估中最常用、最全面的框架。通过这套指标，我们不仅能看到"哪篇文章更好"，还能具体知道**好在哪里、差在哪里**，为后续优化提供明确方向。

2.4.4 评分大模型构建

（1）概念介绍

为了使单 Agent 与 Multi-Agent 的对比结果更加**客观、公正、可量化**，我们引入一个独立的**评分大模型（Judge LLM）**作为评审专家。

这个 Judge LLM 专门负责阅读两篇文章，并严格按照"忠实度、相关性、结构清晰度、专业性、可读性"五个维度进行打分，避免人工评分的主观偏差。

温度设置为 0，确保评分结果稳定、一致。

（2）评分 Prompt

```
judge_llm = ChatOpenAI(temperature=0.0, model="deepseek-chat")

scoring_prompt = """
你是一位严格、公正、专业且经验丰富的金融科技论文评审专家。
现在请你对以下两篇文章进行客观评分，主题为《量子计算对金融行业的影响》。

【文章A - 单 Agent 输出】
{article_a}

【文章B - 多 Agent 输出】
{article_b}

请严格按照以下5个维度分别打分（每个维度满分10分），并给出简要理由：

1. 忠实度（Faithfulness）：文章内容是否忠实于可靠事实，没有幻觉、编造或过时信息。
2. 相关性（Relevancy）：文章是否紧密围绕"量子计算对金融行业的影响"这一主题，没有明显跑题。
3. 结构清晰度（Coherence & Structure）：文章逻辑是否连贯、层次是否分明、结构是否完整（引言、主体、结论）。
4. 专业性（Professionalism & Depth）：术语使用是否准确、专业深度是否足够，分析是否有洞见。
5. 可读性（Readability）：语言是否流畅、自然、通俗易懂，适合非专业读者阅读。

最后给出总分（满分50分）和最终推荐（哪篇文章更好，为什么）。

请严格以以下 JSON 格式输出，不要添加任何多余文字：
{{
```

```
  "faithfulness": xx,
  "relevancy": xx,
  "structure": xx,
  "professionalism": xx,
  "readability": xx,
  "total_score": xx,
  "winner": "A 或 B",
  "reason": "简要总结理由"
}}
"""
```

（3）评分函数实现

```python
def score_articles(article_a: str, article_b: str):
    """
    使用 Judge LLM 对单 Agent 和 Multi-Agent 生成的文章进行评分
    """
    prompt = scoring_prompt.format(article_a=article_a, article_b=article_b)
    result = judge_llm.invoke(prompt)
    return result.content
```

使用示例:

```python
# 执行评分
scores_json = score_articles(
    result_single["output"],
    result_multi
)
print(scores_json)
```

教学小结:

通过这个评分大模型，我们能够获得**结构化、可量化的对比数据**，而不是仅凭主观感受判断“好坏”。

2.4.5 实验结果分析与讨论

如果出现“Multi-Agent 生成内容质量反而低于单 Agent”的情况，应该如何解释?

这是一个非常常见、也很重要的现象，尤其是在初次实验中。

1.可能的原因分析

Prompt 设计不足（最常见原因）

- 每个 Agent 的 `role`、`goal`、`backstory` 描述不够清晰、具体，导致 Agent 无法准确扮演自己的角色。
- Task 的 `description` 和 `expected_output` 写得太模糊，Agent 不知道该输出什么格式、什么深度。

通信与上下文传递问题

- `context` 传递的信息过多或过少，导致后续 Agent 收到的是“噪音”而不是有用信息。

- 顺序执行时，前一个 Agent 的输出质量不高，会直接拖累后面的 Agent（错误累积）。

流程设计不合理

- 简单的 Sequential 流程虽然结构清晰，但缺少反馈循环（Feedback Loop），无法让 Agent 互相纠正。
- 校验 Agent 的 Prompt 设置得过于严格，导致它过度修改甚至破坏了原文的优点。

模型能力与工具限制

- 当前使用的 LLM 在长上下文理解或工具调用上不够强。
- Wikipedia 搜索工具返回的结果质量不高（摘要过短、内容陈旧），导致研究员 Agent 提供的信息本身就有偏差。

"过度分工"效应

- 当任务本身并不复杂时，强行拆分成多个 Agent，反而增加了沟通开销和信息丢失的风险，导致整体质量下降。

2.正确对待态度

Multi-Agent 不是"万能药"，也不是"必然比单 Agent 好"。

它只是在**特定场景**（复杂、多领域、高准确性需求的任务）下，才有可能展现出明显优势。

如果本次实验中 Multi-Agent 表现不如单 Agent，反而说明了一个非常重要的道理：

> **Multi-Agent 的效果高度依赖系统设计质量。**
>
> 好的分工 + 好的 Prompt + 好的流程设计，才能让 1+1 > 2；
>
> 如果设计不当，甚至可能出现 1+1 < 1 的情况。

3.教学小结：

这次对比实验的真正价值不在于"谁赢了"，而在于**让我们看到 Multi-Agent 系统的复杂性。**

它告诉我们：**Multi-Agent 的强大不是自动产生的，而是需要精心设计和不断迭代的。**

这也正是我们学习 Multi-Agent 的核心意义 —— 从"会用"走向"会设计"。

2.5 进阶思考与拓展

到目前为止，我们已经学习了 CrewAI 的核心抽象、通信机制、记忆模型以及基础的 Sequential 流水线模式。这些内容已经能够帮助我们快速搭建一个可运行的多 Agent 系统。

然而，在实际项目中，**很多复杂任务无法通过简单的顺序流水线来完美解决。**例如：

- 需要多轮讨论和动态调整的场景
- 需要多个 Agent 共同决策、达成共识的场景
- 需要系统具备更强鲁棒性和灵活性的场景

这时，我们必须从"基础使用"走向"进阶设计"。 **进阶思考的核心目的**是： 帮助我们突破 CrewAI 默认的"线性协作"限制，掌握更多灵活、高效、接近真实业务场景的设计模式，从而真正把 Multi-Agent 系统应用于复杂生产环境中。

2.5.1 更多灵活设计模式

CrewAI 默认采用的是**流水线模式（Sequential）**，即任务按顺序依次执行。这种模式简单可靠，但对于更复杂的协作场景，显得不够灵活。

除了默认流水线模式，CrewAI 还支持其他更高级的设计模式，下面我们重点介绍**轮值主席模式**和**投票机制**在 CrewAI 中的实现思路。

1. 轮值主席模式（Rotating Chairperson）

模式特点：

一个 Agent 担任"主席"角色，负责主持讨论、分配发言权和总结，其他 Agent 轮流发言、提供意见，主席可以动态切换。

在 CrewAI 中的实现方式：

- 使用 **Hierarchical 流程** + **Manager Agent 动态切换** 的方式近似实现。
- 关键参数：
    - `process=Process.hierarchical`
    - `manager_agent=xxx`（指定当前担任主席的 Agent）
    - 通过多个 Task 的 `context` 和自定义 Prompt，让 Manager Agent 决定下一个"发言"的是哪个 Agent
    - 可在 Manager Agent 的 `backstory` 中加入"轮值主持"的角色设定

适用场景：需要多轮讨论、头脑风暴、方案对比的复杂任务。

2. 投票机制（Voting / Consensus）

模式特点：

多个 Agent 独立完成同一子任务，然后通过投票或加权汇总的方式决定最终输出结果，常用于需要高可靠性的判断场景。

在 CrewAI 中的实现方式：

- 采用**并行 Task + 独立 Judge Task** 的组合模式。
- 关键参数：
    - 多个 Task 设置相同的 `description`，但绑定不同的 Agent（实现并行独立判断）
    - 使用一个专门的 **Judge Agent**（或 Judge Task）来收集所有投票结果
    - 在 Judge Task 的 `description` 中明确投票规则（多数投票、加权投票、带置信度投票等）
    - 通过 `context` 将所有并行 Task 的输出传递给 Judge Task 进行汇总

适用场景：代码审查、方案评估、风险判断、内容审核等需要多人共识的场景。

教学小结：

CrewAI 原生最强的是**流水线模式**和**分级模式（Hierarchical）**，而**轮值主席**和**投票机制**虽然没有内置一键支持，但可以通过合理组合 `Process.hierarchical`、`context` 和 Judge Task 来实现。
真正的灵活性来自于**对核心抽象（Agent、Task、Process）的深度理解和组合使用。**

2.5.2 评估多 Agent 系统的指标

构建完一个 Multi-Agent 系统后，我们不能只看“最终输出结果是否好看”。一个真正优秀的系统，需要从**多个维度**进行全面评估。

评估多 Agent 系统的核心目的是：

不仅要知道“它能不能完成任务”，更要知道“它完成得有多高效、多稳定、多可持续”。

以下是业界常用的**5个核心评估指标**（建议在实际项目中至少跟踪其中3-4个）：

指标	定义	衡量方式	理想趋势	说明
任务完成率	最终输出是否满足初始需求和质量标准	Judge LLM 评分 或 人工评审通过率	接近 100%	最基础的指标
协调效率	通信开销与有效产出的比例	总消息数量 / 最终有用输出数量	越低越好	反映系统是否“话多但事少”
鲁棒性	单个 Agent 失败或异常时系统的容错能力	模拟某个 Agent 异常，观察系统是否能降级	优雅降级或自动恢复	体现系统的稳定性
可扩展性	增加 Agent 数量后系统的性能变化	从 3 个 Agent 扩展到 6 个，观察延迟和质量变化	接近线性增长	决定系统能否应对更大规模任务
整体质量	综合输出质量（忠实度、专业性等）	多维度 Judge LLM 评分	持续提升	最终用户最关心的指标

额外推荐指标（可选）：

- **沟通开销**：平均每个 Task 产生的消息数量
- **成本效率**：完成任务所需的 Token 消耗或 API 调用次数
- **响应时间**：端到端完成任务所需的时间

教学小结：

评估 Multi-Agent 系统时，不能只看“最终文章写得好不好”。

一个优秀的系统应该在**任务完成率、协调效率、鲁棒性、可扩展性**等多个维度上都表现良好。

在实际项目开发中，建议建立一套**多维度评估体系**，定期运行对比实验，不断迭代优化。

2.6 本章总结与未来展望

2.6.1 本章小结

在本章中，我们从**单 Agent 的边界**出发，系统学习了 Multi-Agent 系统的核心概念、设计思路和实战方法。

我们重点掌握了以下内容：

- 单 Agent 在复杂任务中的局限性，以及 Multi-Agent 通过**分工、协作、专业化**实现 1+1 > 2 的核心价值；
- Multi-Agent 特有的**四个额外维度**（控制、知识、目标、行为）以及**社会性与涌现行为**两大核心特性；
- CrewAI 框架的**四个核心抽象**（Agent、Task、Crew、Process）以及通信、记忆、任务流转等关键机制；

- 通过实际项目（量子计算对金融行业影响的短文）完成了**单 Agent vs 多 Agent** 的对比实验，并学会使用 Judge LLM 进行客观评估。

核心收获：

Multi-Agent 不是简单地把多个 LLM 堆在一起，而是**一个系统工程。**它的强大来自于精心设计的分工、协调和涌现智能，而真正的挑战在于如何平衡效率、质量与复杂度。

2.6.2 未来展望

Multi-Agent 技术正处于快速发展阶段，未来将呈现出以下几个重要趋势：

- **Agent 间协议标准化**：类似 MCP（Model Context Protocol）这样的标准协议将逐渐普及，让不同框架、不同模型的 Agent 能够像“插 USB”一样轻松互联，形成真正的生态。
- **Agent 自我组织能力**：未来的系统可能具备**自动发现最优分工、动态调整角色、自我优化流程**的能力，无需人工提前设计全部规则。
- **多模态与跨领域协作**：视觉 Agent、语音 Agent、工具 Agent、代码 Agent 将深度融合，共同完成更复杂的现实任务（如智能科研、自动办公、虚拟团队等）。
- **企业级落地加速**：从内容生成、客服、报告撰写等简单场景，向审批流、供应链管理、科学研究、软件开发等高价值领域快速扩展。

结语：

Multi-Agent 是 Agent 技术从“个体智能”走向“群体智能”的关键一步。

希望通过本章的学习，你不仅掌握了工具，更重要的是建立了**“用团队思维解决复杂问题”** 的系统化思考方式。